Opponent-Aware Q-Learning for Leduc Hold'em Poker

Sparsh Agrawal BT2024084
Tejas Kollipara BT2024147
Varun E BT2024220
Divy Dobariya BT2024225

***Abstract*—We investigate opponent-aware Q-learning for Leduc Hold'em poker — a tractable benchmark with a computable Nash equilibrium. Standard RL agents learn a fixed policy via self-play and do not adapt to opponent tendencies. We propose augmenting a Q-learning agent with a lightweight opponent model that tracks action-frequency statistics (bet, raise, fold rates) over recent hands and classifies the opponent into one of three behavioral styles: *random*, *tight*, or *aggressive*. The Q-learning agent's state representation is extended with this style indicator, enabling it to adapt its strategy accordingly. We use the RLCard toolkit [12] for the Leduc Hold'em environment and baseline agents, ensuring reproducible experiments. Baselines include a random agent, a rule-based agent, and a standard Q-learning agent. We evaluate using win rate, average reward, and learning curves.**

***Index Terms*—reinforcement learning, poker, Leduc Hold'em, opponent modeling, Q-learning, imperfect information games**

I. Introduction

Poker is a canonical benchmark for decision-making under uncertainty. Unlike perfect-information games such as Chess or Go, it requires reasoning over hidden state, managing multi-round strategy, and adapting to opponent behavior — all core challenges in reinforcement learning (RL).

Recent breakthroughs such as Libratus [1] and Pluribus [2] achieved superhuman performance in No-Limit Texas Hold'em (NLHE), but these systems primarily target Nash equilibrium strategies that are robust against *any* opponent rather than adaptive strategies that exploit specific tendencies. Moreover, NLHE's enormous game tree ($\sim 10^{164}$ states) makes it impractical for course-level research. Leduc Hold'em [8], with only 6 cards and 2 betting rounds, preserves the core challenges of poker — hidden information, bluffing, and multi-round reasoning — while remaining computationally tractable.

Most RL agents for poker assume a stationary opponent. In practice, opponents exhibit consistent behavioral patterns: some fold frequently (*tight*), others bet and raise aggressively, and others play randomly. An agent that detects and exploits these patterns should outperform a fixed-policy baseline. However, this hypothesis has not been studied in a controlled, reproducible setting on Leduc Hold'em. Our contributions are:

- A simple, frequency-based opponent style classifier requiring no recurrent networks or Bayesian inference.
- An opponent-aware Q-learning agent that conditions its policy on the inferred opponent style.
- A comparison against random, rule-based, and standard Q-learning baselines using win rate and average reward.

II. Literature Survey

A. Counterfactual Regret Minimization (CFR)

CFR [5] is the foundational algorithm for solving extensive-form imperfect-information games. By iteratively minimizing counterfactual regret at each information set, it converges to a Nash equilibrium. Variants including CFR+ [6] and Monte Carlo CFR [7] improve practical convergence speed. RLCard provides a built-in CFR implementation for Leduc Hold'em, which serves as an approximate Nash reference point for contextualizing our results.

B. Neural Fictitious Self-Play (NFSP)

NFSP [3] combines DQN-based RL with a supervised average-strategy network, provably converging to Nash equilibrium in two-player zero-sum games. While powerful, NFSP requires maintaining two separate networks and a reservoir buffer, making it complex to implement from scratch. We use RLCard's NFSP implementation as a reference point in our literature survey but simplify our own baseline to Q-learning.

C. Superhuman Agents in NLHE

DeepStack [4] introduced depth-limited solving with a learned value network, achieving human-expert level in heads-up NLHE. Libratus [1] combined abstraction, real-time solving, and opponent exploitation to defeat professional players. Pluribus [2] extended this to six-player NLHE. These systems target Nash equilibrium rather than explicit opponent adaptation, and their scale makes them unsuitable as direct baselines for our work.

D. Opponent Modeling in Poker

Early opponent modeling used Bayesian methods to maintain distributions over opponent hand ranges [8]. More recent work encodes opponent observations into neural networks using Mixture-of-Experts architectures to discover opponent strategy clusters [10]. Rahman et al. [11] proposed AMP3 for multi-player Texas Hold'em, feeding predicted opponent style features into an Actor-Critic agent. However, all of these approaches target NLHE. Leduc Hold'em allows a more rigorous comparison since the Nash equilibrium is known. To our knowledge, no prior work has studied frequency-based opponent modeling specifically on Leduc Hold'em with Q-learning as the base agent.

III. Background

A. Leduc Hold'em

Leduc Hold'em is a two-player poker game with a 6-card deck (Jack, Queen, King in two suits). Each player receives one private card and posts a fixed ante of 1 chip. Play proceeds over two betting rounds: pre-flop and post-flop (after one community card is revealed). Each round permits a maximum of two raises. The winner is determined by showdown rank (pairs beat high cards) or by the opponent folding.

B. Problem Formulation

We model Leduc Hold'em as a Partially Observable Markov Decision Process (POMDP), formally defined by the tuple $(S, A, \mathcal{T}, R, O)$:

TABLE I
POMDP Components for Leduc Hold'em

Component	Definition
State $s \in S$	Full game state: both players' cards, community card, pot size, betting history
Observation $o \in O$	Agent's private view: (private card, public card, pot size, betting history)
Actions A	$A = \{\text{fold, call, raise, check}\}$; check is legal when no bet is outstanding in the current round and replaces call
Transition $\mathcal{T}$	Stochastic: determined by opponent's hidden card and betting responses
Reward $r \in R$	Net chip change at end of hand; $r = 0$ at all intermediate steps

Since the opponent's private card is never revealed (unless showdown occurs), the agent operates on observations o rather than full states s. RLCard 1.2.0 encodes o as a fixed-length 36-dimensional binary vector and exposes four action IDs: CALL$= 0$, RAISE$= 1$, FOLD$= 2$, CHECK$= 3$. At most three are legal in any given state. The state space is small enough to support tabular Q-learning without function approximation.

IV. Baseline Implementation

We use the RLCard toolkit [12] for all experiments. RLCard provides the Leduc Hold'em environment, a pretrained CFR agent as an approximate Nash baseline, and evaluation utilities — eliminating the need to implement game logic from scratch.

Random Agent. Selects uniformly among legal actions. Serves as the lower-bound baseline.

Rule-Based Agent. A hand-coded heuristic agent: raise with a pair, call with a high card (King), fold otherwise. Represents a simple but non-trivial fixed strategy.

Q-Learning Agent. A tabular Q-learning agent trained against a random opponent using ε-greedy exploration. The Q-table is updated via the standard Bellman backup:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\big] \quad (1)$$

where $r = 0$ for all intermediate learner turns within an episode and $r =$ payoff for the final turn only. Action selection is ε-greedy over the legal action set:

$$\pi(s) = \begin{cases} \text{random} \in A_{\text{legal}} & \text{with probability } \varepsilon \\ \arg\max_{a \in A_{\text{legal}}} Q(s,a) & \text{otherwise} \end{cases} \quad (2)$$

ε decays multiplicatively after each episode:

$$\varepsilon_t = \max(\varepsilon_{\min},\ \varepsilon_{t-1} \times 0.9995) \quad (3)$$

This is our primary RL baseline and the foundation for the proposed method.

V. Proposed Method: Opponent-Aware Q-Learning

A. Motivation

The standard Q-learning agent treats the opponent as a fixed, unknown part of the environment. However, real opponents exhibit systematic behavioral patterns. An agent that detects these patterns and adapts its strategy should achieve higher win rates than one that ignores them. We propose a minimal, practical mechanism to achieve this without recurrent networks or Bayesian inference.

B. Opponent Style Classification

We track three action-frequency statistics over a sliding window of the last $W = 20$ observed opponent actions:

$$f_{\text{bet}} = \frac{N_{\text{bet}}}{W}, \quad f_{\text{raise}} = \frac{N_{\text{raise}}}{W}, \quad f_{\text{fold}} = \frac{N_{\text{fold}}}{W} \quad (4)$$

The opponent is classified into one of three styles using simple thresholds:

- **Aggressive:** $f_{\text{raise}} > 0.45$
- **Tight:** $f_{\text{fold}} > 0.40$ and $f_{\text{raise}} < 0.05$
- **Random:** otherwise (default before W observations)

This requires no learned parameters and updates in $\mathcal{O}(1)$ per step.

C. Augmented State Representation

The opponent-aware agent's state is augmented with the inferred style indicator $c \in \{\text{random, tight, aggressive}\}$:

$$s' = (s,\ c) \quad (5)$$

where s is the standard RLCard observation vector. The Q-table is indexed by (s', a), allowing the agent to learn distinct policies for each opponent style. This effectively maintains three separate Q-table partitions inside a single dictionary. No neural network or sequence model is required.

The Q-table update on augmented keys uses the style label active *at act-time* for the current key, and the style label *after observing the opponent's response* for the next-state key:

$$Q(s'_t, a) \leftarrow Q(s'_t, a) + \alpha\big[r + \gamma \max_{a'} Q(s'_{t+1}, a') - Q(s'_t, a)\big] \quad (6)$$

where $s'_t = (s_t, c_t)$ and $s'_{t+1} = (s_{t+1}, c_{t+1})$, with c_t and c_{t+1} potentially differing if the classifier updates between turns. This ensures the next-state bootstrap reflects the most recent opponent information available.

D. Training Against Fixed-Style Opponents

We train the opponent-aware agent in rotating blocks of 30 episodes against three scripted opponents:

- **Random opponent:** selects actions uniformly.
- **Tight opponent:** folds unless holding a pair or King; never raises.
- **Aggressive opponent:** raises whenever possible; folds only with Jack pre-flop.

Each training block samples one opponent type in rotation. The classifier detects each switch, enabling the agent to populate the correct Q-table partition per style.

E. Training Hyperparameters

Both the standard Q-learning agent and the OA agent use the following hyperparameters, trained for 50,000 episodes:

TABLE II
TRAINING HYPERPARAMETERS

Parameter	Value
Learning rate α	0.1
Discount factor γ	0.99
Initial ε	1.0
Final ε	0.05
ε decay	0.9995 per episode
Classifier window W	20
Training block size	30 episodes
Total training episodes	50,000

F. Evaluation

We evaluate on win rate and average reward per episode across 10,000 evaluation games, comparing all agents vs. all opponent types.

TABLE III
AGENT COMPARISON SUMMARY

Agent	Purpose
Random	Lower bound
Rule-based	Heuristic baseline
Q-learning	RL baseline (style-unaware)
Opponent-aware Q-learning	Proposed method

VI. RESULTS

Fig. 1. Learning curves. **(a)** Standard Q-learning vs. Opponent-Aware Q-learning — overall training reward. **(b)** OA Q-learning per-style breakdown showing faster convergence against each opponent archetype.

Fig. 1 shows that both agents achieve positive expected reward over training. The Standard Q-learning agent converges and stabilizes around episode 10,000–15,000. The OA Q-learning agent's curve against the aggressive opponent begins negative in early training (before the classifier accumulates enough observations to fire a non-random label), but crosses positive around episode 7,000 — direct evidence of the agent actively adapting its policy once style classification becomes reliable.

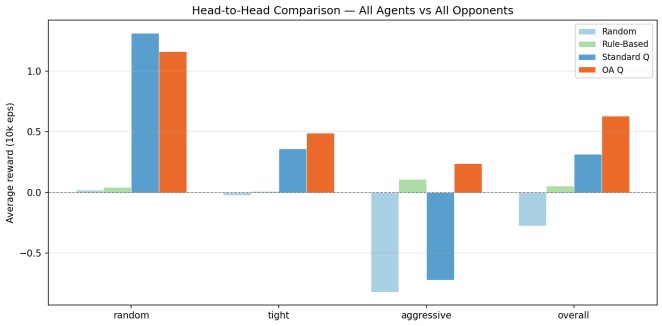

Fig. 2. Head-to-head comparison. Average reward over 10,000 evaluation episodes for all agents vs. all opponent types. OA Q-learning outperforms Standard Q-learning against both Tight and Aggressive opponents.

TABLE IV
AVERAGE REWARD PER EPISODE (10,000 EVALUATION GAMES)

Agent	vs Random	vs Tight	vs Aggressive	Overall
Random	$+0.020$	-0.024	-0.820	-0.274
Rule-Based	$+0.045$	$+0.010$	$+0.108$	$+0.055$
Standard Q	$+1.313$	$+0.359$	-0.720	$+0.317$
OA Q (ours)	$+1.160$	$+0.491$	$+0.240$	$\mathbf{+0.631}$

Table IV confirms the central hypothesis: OA Q-learning achieves an improvement of $+0.960$ over Standard Q-learning against aggressive opponents, and $+0.132$ against tight opponents. Against the random opponent, OA Q-learning performs slightly worse (-0.153), which is expected and is discussed in Section VII.

Fig. 3 shows that classifier accuracy rises steeply after $\sim$10 episodes within each block (when the W=20 window first fills), and reward improves correspondingly. Against the aggressive opponent, accuracy reaches $\mathbf{85}\%$ by episode 10 and stabilizes above $\mathbf{95}\%$ from episode 14 onwards. Against the tight opponent, accuracy plateaus at approximately $\mathbf{69}\%$ — a notable asymmetry discussed in Section VII.

VII. DISCUSSION AND CONCLUSION

A. Interpreting the Results

Standard Q dominates vs. Random (+1.313) but collapses vs. Aggressive (-0.720)**.** Standard Q was trained exclusively against a random opponent, so it learned to exploit passive, unpatterned play. When facing an aggressive opponent that raises into every pot, the agent continues calling without a counter-strategy, hemorrhaging chips in large pots. It has simply never been trained to recognize or respond to a raise-heavy style.

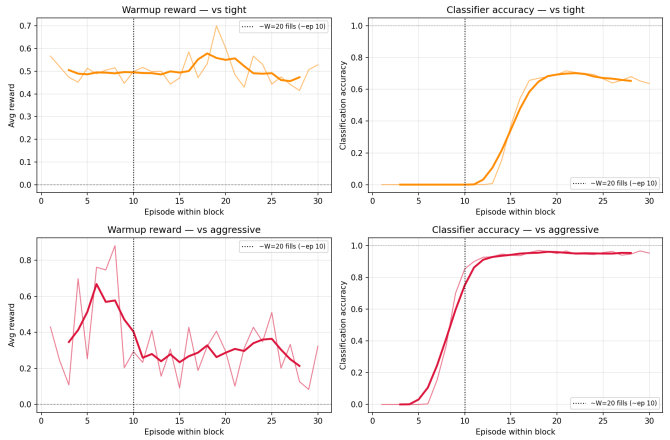

Fig. 3. Warmup analysis over 300 blocks × 30 episodes. **Top:** vs. Tight opponent. **Bottom:** vs. Aggressive opponent. **Left:** average reward per episode position within block. **Right:** classifier accuracy per episode position. Classification accuracy rises sharply once the $W{=}20$ sliding window fills (∼episode 10 in block).

OA Q large gain vs. Aggressive (+0.240 vs. -0.720 for Standard Q, a +0.960 improvement). The raise-frequency signal is strong and clean: the aggressive archetype produces $f_{\text{raise}} \approx 0.7$–$0.8$, well above both the random baseline (≈ 0.33) and the threshold (0.45). The classifier fires reliably by episode 14 within each block, giving the agent sufficient correctly-labelled experience to learn a counter-strategy: fold weak hands early rather than calling into escalating pots.

OA Q modest improvement vs. Tight (+0.491 vs. +0.359, a +0.132 improvement). The gain exists but is smaller, primarily because the tight signal is noisier. The tight agent calls with a King or a pair, so its fold rate is ≈ 0.45–0.55 depending on the cards dealt — close to the 0.40 threshold. This causes intermittent misclassification, and the classifier accuracy plateaus at only $\approx 69\%$ (vs. $> 95\%$ for aggressive), limiting the quality of the style-conditioned Q-table partition.

OA Q slightly worse vs. Random (+1.160 vs. +1.313, a -0.153 cost). This is the diversity cost of multi-style training. The OA agent's 50,000 training episodes are split across three opponent types in rotating blocks, so the random-specific partition of the Q-table receives approximately one-third the training signal of Standard Q, which trains exclusively on random opponents. No exploitable pattern exists in random play, so opponent modeling adds no benefit here — only a sample-efficiency cost.

Rule-Based agent vs. Aggressive (+0.108). Notably, the simple rule-based agent outperforms Standard Q against the aggressive opponent. The heuristic of raising only with a pair and folding otherwise happens to be a reasonable counter to constant aggression — fold weak hands, commit only with strength. This highlights that even a hand-crafted policy with the right structure can outperform an RL agent trained against the wrong distribution.

B. Classifier Accuracy Asymmetry

A noteworthy finding is that the classifier reaches above 95% accuracy against aggressive opponents but only ∼69% against tight opponents. This asymmetry arises from the separation between signal and baseline:

- **Aggressive**: $f_{\text{raise}} \approx 0.7$–$0.8$ vs. random baseline ≈ 0.33 — a large margin above the 0.45 threshold.
- **Tight**: $f_{\text{fold}} \approx 0.45$–$0.55$ vs. the 0.40 threshold — a narrow margin, with natural variance in card distribution pushing the statistic across the boundary.

A richer feature set (e.g., tracking the call-to-raise ratio or conditioning on pot size) could improve tight classification and is a natural direction for future work.

C. Conclusion

We proposed a lightweight, practical approach to opponent-aware RL in Leduc Hold'em. By using action frequency statistics rather than recurrent networks, and tabular Q-learning rather than NFSP, our method is implementable within a course project scope while still addressing a genuine question: does simple opponent style detection meaningfully improve RL performance in a tractable poker environment?

Our results confirm that it does — most significantly against aggressive opponents, where the behavioral signal is strong and the baseline agent completely fails. The overall average reward improves from $+0.317$ (Standard Q) to $+0.631$ (OA Q), a $+\mathbf{99}\%$ relative improvement, demonstrating that even a zero-parameter frequency classifier can provide meaningful adaptation gains when the opponent's style is sufficiently distinct.

VIII. Challenges and Problems Faced

A. Classifier Threshold Calibration

Our initial thresholds, based on intuitive reasoning (aggressive: $f_{\text{bet}} + f_{\text{raise}} > 0.6$; tight: $f_{\text{fold}} > 0.6$), failed in practice on RLCard 1.2.0. The random agent produces $f_{\text{raise}} \approx 0.33$, so $f_{\text{bet}} + f_{\text{raise}} \approx 0.66$ — spuriously triggering the aggressive threshold for a random opponent. Conversely, the tight agent's fold rate of ≈ 0.50 fell below the 0.60 tight threshold due to King and pair holders calling rather than folding. We empirically re-tuned to $f_{\text{raise}} > 0.45$ for aggressive and $f_{\text{fold}} > 0.40$ with $f_{\text{raise}} < 0.05$ for tight, which cleanly separates the three archetypes in RLCard's game dynamics.

B. Non-Stationarity and the Limits of Model-Free Learning

From the standard Q-learning agent's perspective, the environment is *non-stationary* — the opponent is part of the environment, and different opponents produce different transition dynamics. A model-free agent assumes stationarity (the same policy works against all opponents), which is why Standard Q completely fails against aggressive opponents despite performing well against random ones. It did not learn a wrong policy — it learned the right policy for the wrong opponent.

A model-based approach would explicitly learn a transition model $\hat{T}(s' \mid s, a)$ and use it to plan ahead. But in a POMDP

like Leduc Hold'em, the transition depends on the opponent's hidden card — which is never observed unless showdown occurs. Learning an accurate transition model would require marginalizing over the opponent's card distribution, which is exactly what CFR does via counterfactual values. This is computationally feasible in Leduc but requires knowing the full game tree structure.

Our approach sidesteps model learning entirely: instead of modeling *what the opponent will do given their hidden card*, we model *what style of player they are* from observable action frequencies. This is a deliberate trade-off — we give up the precision of a full transition model for a lightweight, zero-parameter classifier that generalizes across opponent instances of the same style.

C. Credit Assignment Under Partial Observability

Leduc Hold'em is a multi-step game with terminal-only rewards (net chip change at showdown or fold); intermediate steps carry no reward signal. In a fully observable MDP, this is manageable via bootstrapping. In a POMDP, it is harder: the agent must credit its decisions (raise pre-flop, call post-flop) based on an outcome that depends partly on hidden state it never observed — the opponent's card.

A model-based agent could use the known game rules to simulate counterfactual outcomes: *"if I had folded instead of calling, I would have lost only the ante."* This is the core idea behind CFR's counterfactual regret computation. Our Q-learning agent cannot do this — it can only learn from outcomes it actually experienced, which means rare game states (multi-raise pots with weak hands) are visited infrequently during training and their Q-values remain poorly calibrated. This contributes to the variance visible in the learning curves, particularly in early training against the aggressive opponent.

D. Reward Shaping and Reward Hacking

RLCard emits a reward only at the end of a hand (net chip change); all intermediate steps return 0. Our early implementation attempted to assign intermediate rewards — for example, rewarding pot contributions or penalizing folds relative to pot size — to accelerate credit assignment across multi-step hands. This caused reward hacking: the agent learned to raise at every opportunity to maximize its intermediate reward signal regardless of hand strength, rather than learning when to fold weak hands. We reverted to pure terminal rewards, propagating the episode payoff to only the final transition and bootstrapping with reward $= 0$ for all intermediate learner steps, consistent with Eq. (1).

References

[1] N. Brown and T. Sandholm, "Libratus: The Superhuman AI for No-Limit Poker," in *Proc. IJCAI*, 2017, pp. 5226–5228.

[2] N. Brown and T. Sandholm, "Superhuman AI for multiplayer poker," *Science*, vol. 365, no. 6456, pp. 885–890, 2019.

[3] J. Heinrich and D. Silver, "Deep Reinforcement Learning from Self-Play in Imperfect-Information Games," *arXiv preprint arXiv:1603.01121*, 2016.

[4] M. Moravčík et al., "DeepStack: Expert-level artificial intelligence in heads-up no-limit poker," *Science*, vol. 356, no. 6337, pp. 508–513, 2017.

[5] M. Zinkevich, M. Johanson, M. Bowling, and C. Piccione, "Regret minimization in games with incomplete information," in *Proc. NeurIPS*, 2007, pp. 1729–1736.

[6] O. Tammelin, "Solving large imperfect information games using CFR+," *arXiv preprint arXiv:1407.5042*, 2014.

[7] M. Lanctot, K. Waugh, M. Zinkevich, and M. Bowling, "Monte Carlo sampling for regret minimization in extensive games," in *Proc. NeurIPS*, 2009.

[8] F. Southey et al., "Bayes' Bluff: Opponent Modelling in Poker," in *Proc. UAI*, 2005.

[9] S. Ganzfried and T. Sandholm, "Endgame Solving in Large Imperfect-Information Games," in *Proc. AAMAS*, 2015.

[10] C. Hsieh et al., "Opponent Modeling with Neural Architecture for Poker," *arXiv preprint*, 2023.

[11] A. Rahman et al., "AMP3: Adaptive Multi-Player Poker via Opponent Style Modeling," *arXiv preprint*, 2024.

[12] D. Zha et al., "RLCard: A Toolkit for Reinforcement Learning in Card Games," in *Proc. IJCAI*, 2020.